

☒ Tank Arena

Juego de **batalla de tanques top-down** multijugador (1–4 jugadores), desarrollado para la materia de **Sistemas de Cómputo Paralelo y Distribuido** (Tercer Parcial).

Dos aplicaciones nativas independientes —una para **Linux** y otra para **Windows**— que comparten **un solo código base en C** y se comunican entre sí por red.

☒ La idea

Tanques vistos desde arriba en arenas con paredes y obstáculos. Los tanques se mueven, giran, disparan, se cubren y explotan. Soporta de 1 a 4 jugadores; los huecos se rellenan con tanques controlados por IA, así que la partida funciona sin importar cuánta gente esté conectada.

Modos de juego (escala 1–4)

Jugadores	Modos
1	1 humano vs IA
2	1v1, o 2 cooperando vs IA
3	Todos contra todos, o 2 humanos + 1 IA
4	Todos contra todos (FFA) , 2v2 por equipos, o 2 humanos vs 2 IA

☒ Requisitos obligatorios y cómo se cumplen

Requisito	Implementación
Equipo máx. 4 personas	Juego de 1–4 jugadores
Código único multiplataforma en C	Capa de abstracción con <code>#ifdef _WIN32</code> / <code>#ifdef __linux__</code>
Multihilo + Mutex/Semáforos	Hilo de red, hilo de juego, hilo de chat, mutex protege el estado compartido (productor-consumidor)
Sockets + protocolo propio	Protocolo propio: cabecera (tipo + longitud) y mensajes <code>JOIN</code> , <code>INPUT</code> , <code>STATE</code> , <code>CHAT</code> , <code>PING</code>
Chat general y por equipos	Chat en GTK; el host enruta y filtra por equipo o difunde a todos
Juego o simulación interactiva	Juego interactivo
GUI con GTK + Glade	Glade para menús/lobby/chat/HUD; juego dibujado en <code>GtkDrawingArea</code> con Cairo

☒ Arquitectura: host autoritativo (cliente-servidor)

- Un nodo **hospeda** la partida (servidor); los demás son **clientes**.
- **Cliente**: captura los controles del jugador, los envía y dibuja el estado recibido.
- **Host**: simula todo (movimiento, colisiones, disparos, daño, IA) y difunde el estado ~30 veces por segundo.
- Como solo el host calcula colisiones e impactos, **nunca hay dos verdades** entre pantallas.

Hilos y sincronización

- **Hilo de red**: recibe mensajes y los encola.
- **Hilo de juego**: consume la cola y actualiza el estado.
- **Hilo principal (GTK)**: dibuja el estado.
- ☒ Un **mutex/semáforo** protege la cola y el estado del juego (exclusión mutua productor-consumidor).
- ⚠ **GTK no es thread-safe**: solo el hilo principal toca widgets. Los hilos de red/juego envían actualizaciones a la GUI vía `g_idle_add`.

Protocolo propio (Linux ↔ Windows)

Como una app de Linux habla con una de Windows, el protocolo debe ser independiente de plataforma:

- **Orden de bytes de red** (`htons` / `htonl`) o protocolo de texto.
- **No enviar `struct` crudos** (el *padding* difiere entre compiladores/SO).
- **Cabecera de longitud fija + cuerpo**, parseado igual en ambos lados.

🔧 Puntos extra (décimas)

1. **Tolerancia a fallos real**: reconexión automática, persistencia del estado en disco para recuperar la partida, y/o **migración de host** (algoritmo Bully) si el host se cae.
2. **Visualización didáctica del paralelismo** 📖: panel en GTK que muestra en vivo los hilos activos, el mutex bloqueado/libre, el tamaño de la cola de mensajes y el reparto de la IA.
3. **Animaciones fluidas**: interpolación del movimiento, explosiones animadas, giro de torreta.

🔧 Tecnología

- **Lenguaje**: C
- **GUI**: GTK 3 + Glade (diseño visual) + **Cairo** (canvas del juego)
- **Red**: Sockets — Winsock (Windows) / POSIX (Linux), tras una capa con `#ifdef`
- **Hilos**: Win32 threads (Windows) / `pthread`s (Linux), tras la misma capa
- **Build**: CMake → se abre en **Visual Studio** (Windows) y compila con `gcc` / `make` (Linux)
- **Sonido (opcional, no está en el rubro)**: [miniaudio](#) si hay tiempo

⚠ **Setup más latoso**: compilar GTK dentro de Visual Studio (MSVC) en Windows. Rutas que funcionan: **vcpkg** (integra GTK con VS) o **MSYS2/MinGW**. Que una persona lo resuelva el día 1 antes de que todos avancen.

🔧 Plan de trabajo

Paso	Qué	Cuándo
1	Setup crítico : GTK compilando en Windows y Linux + ventana con Glade + capa <code>#ifdef</code>	Día 1
2	Base: un tanque que se mueve/gira en el canvas (Cairo) + mapa con colisiones	Día 2
3	🔧 Red + protocolo + hilos + mutex : dos apps (Linux↔Windows) ven los tanques moverse	Días 3–4
4	Combate + chat general/equipos + escalar a 4 + desconexión	Días 5–6
5	IA + modos (FFA / 2v2 / vs IA)	Día 7
6	🔧 Décimas extra : panel didáctico, animaciones, tolerancia a fallos	Resto

📌 **Regla de oro**: primero que las dos apps se comuniquen (paso 3); las décimas extra van al final. Lo que se evalúa es el multijugador funcionando.

🔧 Evaluación

- Autoevaluación: **33%**
- Coevaluación (compañeros): **33%**
- Evaluación del profesor: **34%**